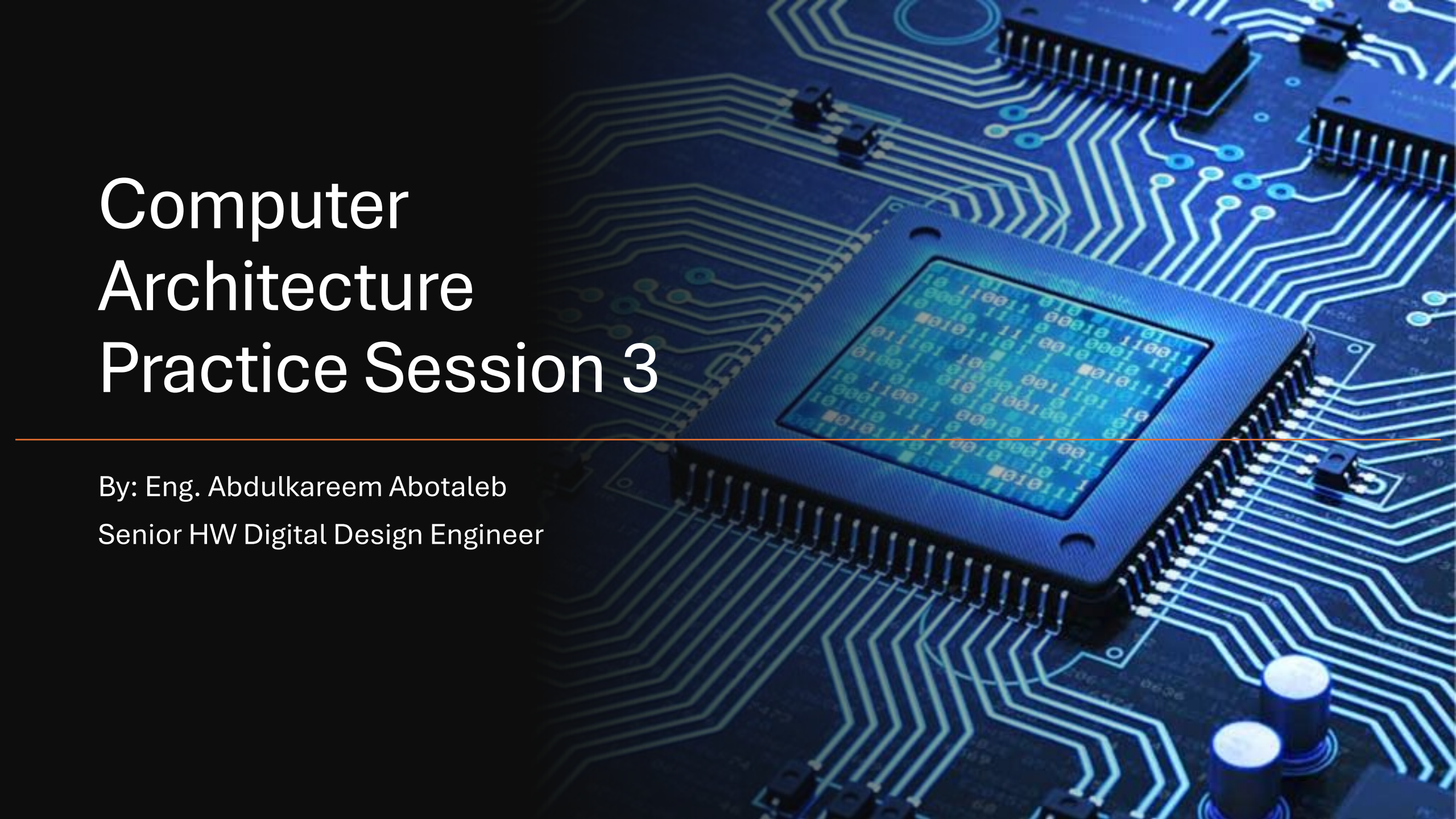
Computer Architecture Practice Session 3



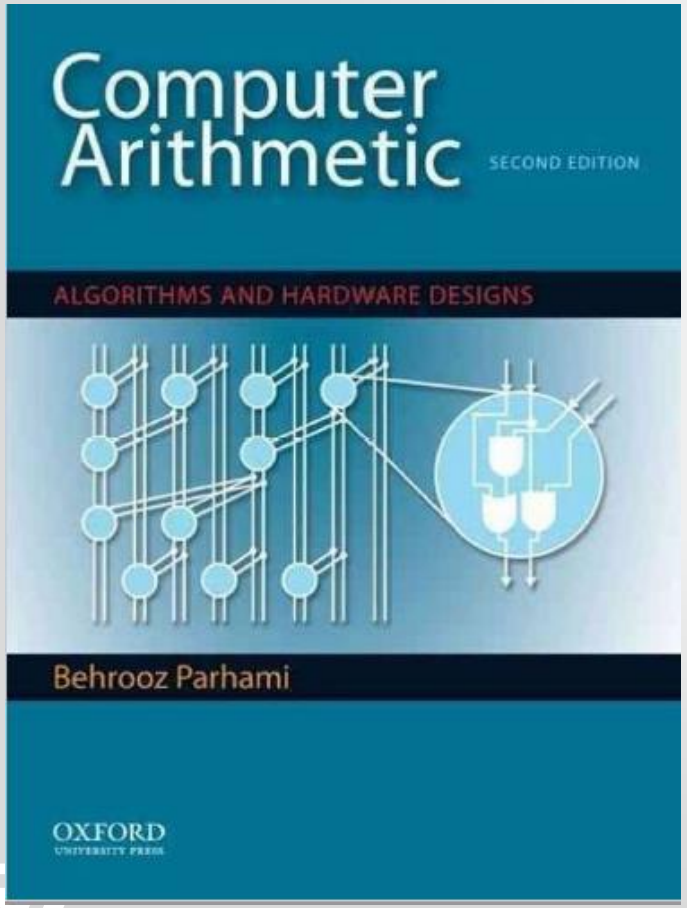
By: Eng. Abdulkareem Abotaleb

Senior HW Digital Design Engineer

Agenda

- # 1. Multiplier
- # 2. Divider
- # 3. Implementation
- # 4. Synthesis

1. Multiplier



life.augmented

Agenda

9 Basic Multiplication Schemes

- # 9.1 Shift/Add Multiplication Algorithms
- # 9.3 Basic Hardware Multipliers
- # 9.4 Multiplication of Signed Numbers

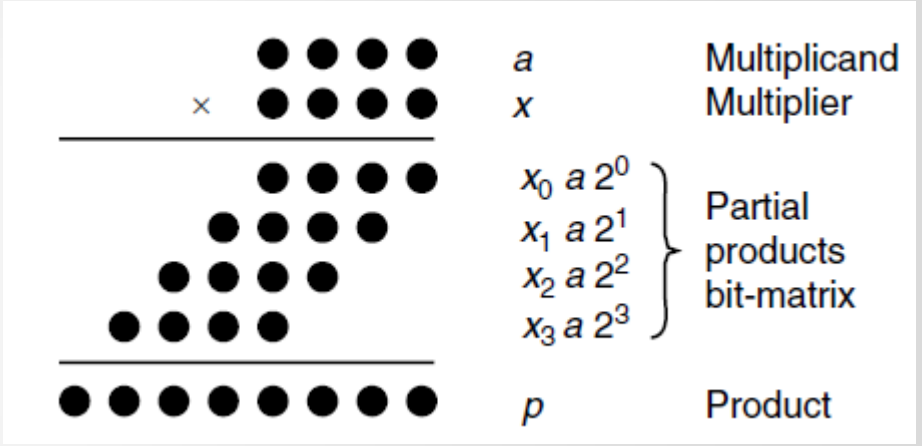
11 Tree and Array Multipliers

- # 11.1 Full-Tree Multipliers

9.1 Shift/Add Multiplication Algorithms

Calculate the partial products:

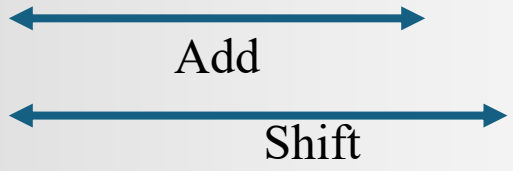
- Since xj is in {0, 1}, each term xja is either 0 or a.
- Thus, the problem of binary multiplication reduces to adding a set of numbers, each of which is 0 or a shifted version of the multiplicand a.
- *The partial product will be:*
  $p(0) = x_0\, a\, 2^0$
  $p(1) = p(0) + x_1\, a\, 2^1$
  $p(2) = p(1) + x_2\, a\, 2^2$
  $p(3) = p(2) + x_3\, a\, 2^3$
- *Then,* $p = p(3) = x_0\, a\, 2^0 + x_1\, a\, 2^1 + x_2\, a\, 2^2 + x_3\, a\, 2^3$



9.1 Shift/Add Multiplication Algorithms

Calculate the partial products:

- $p(j+1) = (p(j) + x_j a\ 2^k)2^{-1}$



with $p(0) = 0$ and $p(k) = p$.

- After k iterations, the preceding recurrence leads to

$$p(k) = a\ x + p(0)2^{-k}$$

(a) Right-shift algorithm

a		1	0	1	0			
x		1	0	1	1			
$p^{(0)}$		0	0	0	0			
$+x_0 a$		1	0	1	0			
$2p^{(1)}$	0	1	0	1	0			
$p^{(1)}$		0	1	0	1	0		
$+x_1 a$		1	0	1	0			
$2p^{(2)}$	0	1	1	1	1	0		
$p^{(2)}$		0	1	1	1	1	0	
$+x_2 a$		0	0	0	0			
$2p^{(3)}$	0	0	1	1	1	1	0	
$p^{(3)}$		0	0	1	1	1	1	0
$+x_3 a$		1	0	1	0			
$2p^{(4)}$	0	1	1	0	1	1	1	0
$p^{(4)}$		0	1	1	0	1	1	1	0

(b) Left-shift algorithm

a				1	0	1	0	
x				1	0	1	1	
$p^{(0)}$				0	0	0	0	
$2p^{(0)}$			0	0	0	0	0	
$+x_3 a$				1	0	1	0	
$p^{(1)}$			0	1	0	1	0	
$2p^{(1)}$		0	1	0	1	0	0	
$+x_2 a$				0	0	0	0	
$p^{(2)}$			0	1	0	1	0	0
$2p^{(2)}$		0	1	0	1	0	0	0
$+x_1 a$				1	0	1	0	
$p^{(3)}$		0	1	1	0	0	1	0
$2p^{(3)}$	0	1	1	0	0	1	0	0
$+x_0 a$				1	0	1	0	
$p^{(4)}$	0	1	1	0	1	1	1	0

9.1 Shift/Add Multiplication Algorithms

Example: 4-bit Multiplier:

$p(j+1) = (p(j) + x_j a\, 2^k)2^{-1}$, with $p(0) = 0$ and $p(k) = p$.

- $p(1) = (0 + x_0 a\, 2^4)* 2^{-1} = x_0 a\, 2^3$
- $p(2) = ((x_0 a\, 2^3) + x_1 a\, 2^4)* 2^{-1}$
  $= x_0 a\, 2^2 + x_1 a\, 2^3$
- $p(3) = ((x_0 a\, 2^2 + x_1 a\, 2^3) + x_2 a\, 2^4)* 2^{-1}$
  $= x_0 a\, 2^1 + x_1 a\, 2^2 + x_2 a\, 2^3$
- $p(4) = ((x_0 a\, 2^1 + x_1 a\, 2^2 + x_2 a\, 2^3) + x_3 a\, 2^4)* 2^{-1}$
  $= x_0 a\, 2^0 + x_1 a\, 2^1 + x_2 a\, 2^2 + x_3 a\, 2^3$

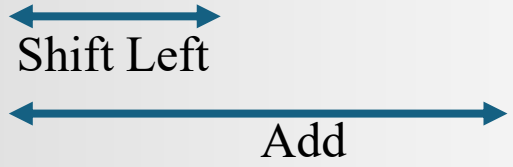
(a) Right-shift algorithm

a			1	0	1	0		
x			1	0	1	1		
$p^{(0)}$		0	0	0	0			
$+x_0 a$		1	0	1	0			
$2p^{(1)}$	0	1	0	1	0			
$p^{(1)}$		0	1	0	1	0		
$+x_1 a$		1	0	1	0			
$2p^{(2)}$	0	1	1	1	1	0		
$p^{(2)}$		0	1	1	1	1	0	
$+x_2 a$		0	0	0	0			
$2p^{(3)}$	0	0	1	1	1	1	0	
$p^{(3)}$		0	0	1	1	1	1	0
$+x_3 a$		1	0	1	0			
$2p^{(4)}$	0	1	1	0	1	1	1	0
$p^{(4)}$		0	1	1	0	1	1	1	0

9.1 Shift/Add Multiplication Algorithms

Calculate the partial products:

- $p(j+1) = 2*p(j) + x_{k-j-1}*a$



with $p(0) = 0$ and $p(k) = p$.

- After k iterations, the preceding recurrence leads to

$$p(k) = a\,x + p(0)2^k$$

(a) Right-shift algorithm

a			1	0	1	0		
x			1	0	1	1		
$p^{(0)}$			0	0	0	0		
$+x_0a$			1	0	1	0		
$2p^{(1)}$	0	1	0	1	0			
$p^{(1)}$		0	1	0	1	0		
$+x_1a$		1	0	1	0			
$2p^{(2)}$	0	1	1	1	1	0		
$p^{(2)}$		0	1	1	1	1	0	
$+x_2a$		0	0	0	0			
$2p^{(3)}$	0	0	1	1	1	1	0	
$p^{(3)}$		0	0	1	1	1	1	0
$+x_3a$		1	0	1	0			
$2p^{(4)}$	0	1	1	0	1	1	1	0
$p^{(4)}$		0	1	1	0	1	1	1	0

(b) Left-shift algorithm

a				1	0	1	0	
x				1	0	1	1	
$p^{(0)}$				0	0	0	0	
$2p^{(0)}$			0	0	0	0	0	
$+x_3a$				1	0	1	0	
$p^{(1)}$			0	1	0	1	0	
$2p^{(1)}$		0	1	0	1	0	0	
$+x_2a$				0	0	0	0	
$p^{(2)}$		0	1	0	1	0	0	
$2p^{(2)}$	0	1	0	1	0	0	0	
$+x_1a$				1	0	1	0	
$p^{(3)}$		0	1	1	0	0	1	0
$2p^{(3)}$	0	1	1	0	0	1	0	0
$+x_0a$				1	0	1	0	
$p^{(4)}$	0	1	1	0	1	1	1	0

9.1 Shift/Add Multiplication Algorithms

Example: 4-bit Multiplier:

$p(j+1) = 2*p(j) + x_{k-j-1}*a$, with $p(0) = 0$ and $p(k) = p$.

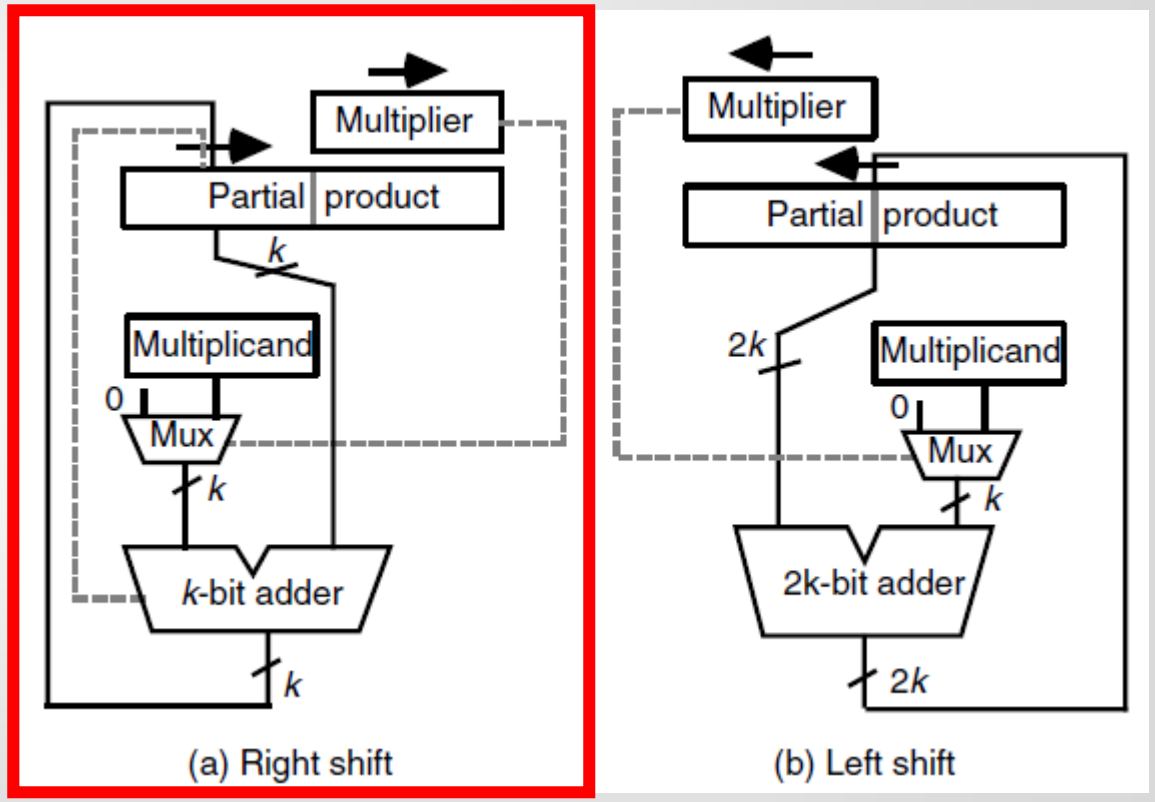
- $p(1) = 0 + x_3 a = x_3 a$
- $p(2) = 2*(x_3 a) + x_2 a = x_3 a\, 2^1 + x_2 a$
- $p(3) = 2*(x_3 a\, 2^1 + x_2 a) + x_1 a$
  $= x_3 a\, 2^2 + x_2 a\, 2^1 + x_1 a$
- $p(4) = 2*(x_3 a\, 2^2 + x_2 a\, 2^1 + x_1 a) + x_0 a$
  $= x_3 a\, 2^3 + x_2 a\, 2^2 + x_1 a\, 2^1 + x_0 a$

(b) Left-shift algorithm

a					1	0	1	0
x					1	0	1	1
$p^{(0)}$					0	0	0	0
$2p^{(0)}$				0	0	0	0	0
$+x_3a$					1	0	1	0
$p^{(1)}$				0	1	0	1	0
$2p^{(1)}$			0	1	0	1	0	0
$+x_2a$					0	0	0	0
$p^{(2)}$			0	1	0	1	0	0
$2p^{(2)}$		0	1	0	1	0	0	0
$+x_1a$					1	0	1	0
$p^{(3)}$		0	1	1	0	0	1	0
$2p^{(3)}$	0	1	1	0	0	1	0	0
$+x_0a$					1	0	1	0
$p^{(4)}$	0	1	1	0	1	1	1	0

9.3 Basic Hardware Multipliers

- $p(j+1) = (p(j) + x_j a\, 2^k)2^{-1}$
- The next bit of the multiplier is always available at the right end of the x register and is used to select 0 or a for the addition.
- Addition and shifting can be performed in 2 separate cycles or in 2 subcycles within the same clock cycle.
- The addition and shifting as a single operation by connecting the ith sum output of the adder to the $(k+i-1)^{th}$ bit of the partial product register and the adder's carry-out to bit 2k−1.



9.4 Multiplication of Signed Numbers

To multiply signed values with complement representations:

- Indirect Multiplication:
  Complement the negative operand(s), multiply unsigned values, and then complement the result if only one operand was complemented at the outset.
- Direct Multiplication:

1- To support a negative 2's-complement multiplicand:

- The bit-at-a-time [Right] algorithm can work directly.
- Each $x_j a$ term will be a 2's-complement number and the sum will be correctly accumulated if we use sign-extended values during the addition process.

a			1	0	1	1	0		
x			0	1	0	1	1		
$p^{(0)}$			0	0	0	0	0		
$+x_0 a$			1	0	1	1	0		
$2p^{(1)}$	1	1	0	1	1	0			
$p^{(1)}$		1	1	0	1	1	0		
$+x_1 a$			1	0	1	1	0		
$2p^{(2)}$	1	1	0	0	0	1	0		
$p^{(2)}$		1	1	0	0	0	1	0	
$+x_2 a$			0	0	0	0	0		
$2p^{(3)}$	1	1	1	0	0	0	1	0	
$p^{(3)}$		1	1	1	0	0	0	1	0
$+x_3 a$			1	0	1	1	0		
$2p^{(4)}$	1	1	0	0	1	0	0	1	0
$p^{(4)}$		1	1	0	0	1	0	0	1	0
$+x_4 a$			0	0	0	0	0		
$2p^{(5)}$	1	1	1	0	0	1	0	0	1	0
$p^{(5)}$		1	1	1	0	0	1	0	0	1	0

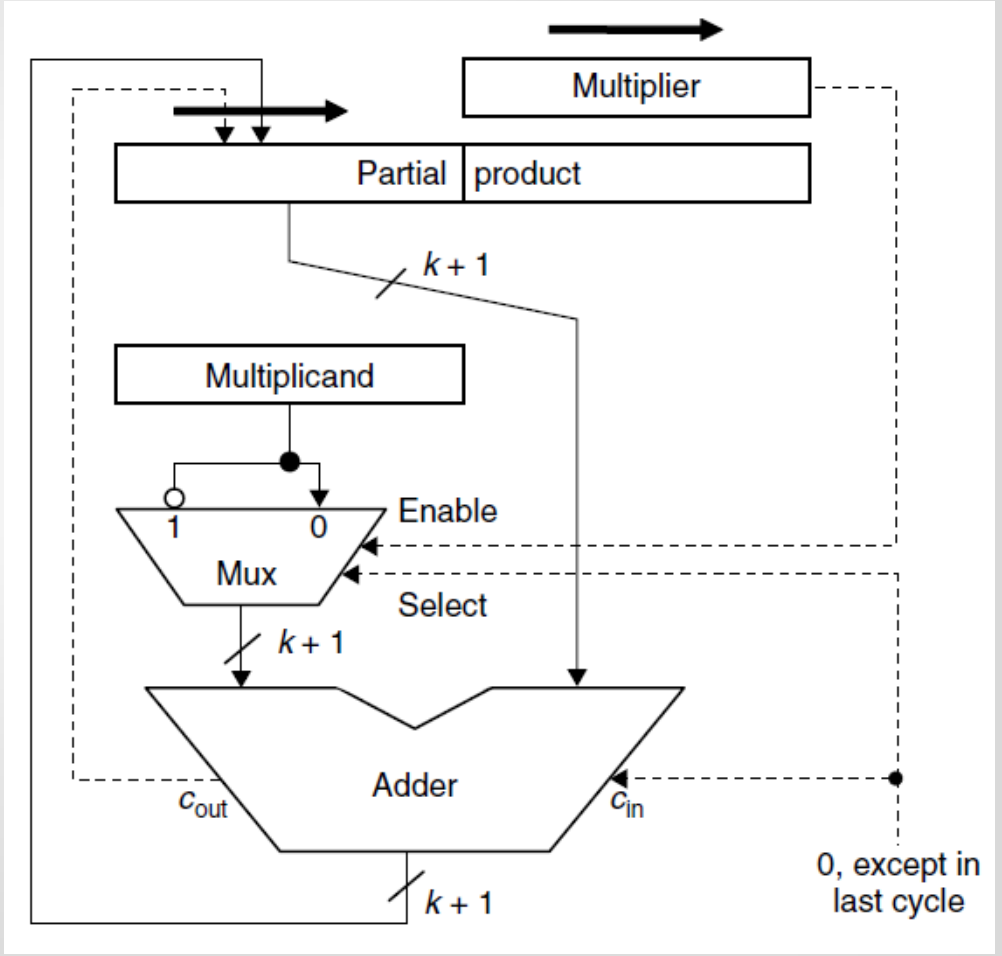
9.4 Multiplication of Signed Numbers

- Direct Multiplication:

2- To support a negative 2's-complement multiplier:

  - The bit-at-a-time [Right] algorithm should be modified to subtract the $x_{k-1}a$ in the last cycle.
  - Multiplicand is added to the partial product in all but the final cycle, a subtraction is performed by choosing the complement of the multiplicand and inserting a carry-in of 1.

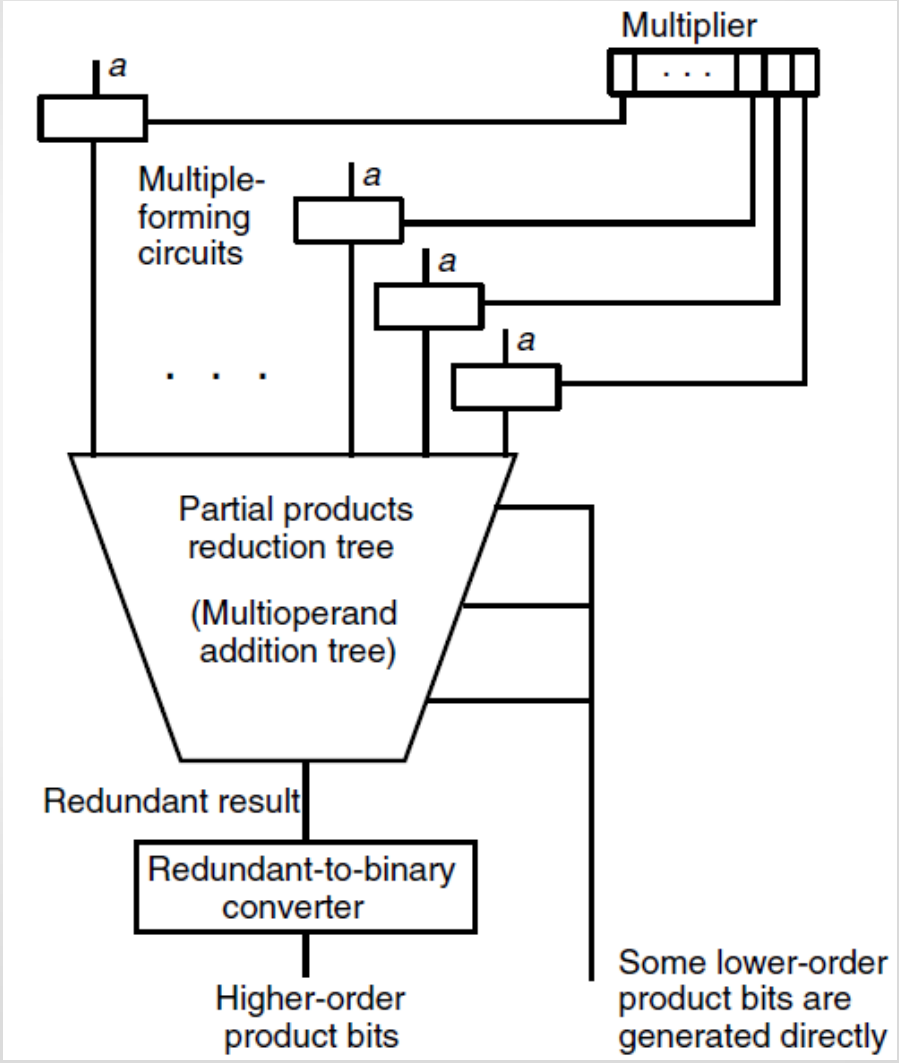
- 2's-complement directly Multiplication with left shifts becomes even **less competitive** as the adder will be more complex (slower additions).



11.1 Full-Tree Multipliers

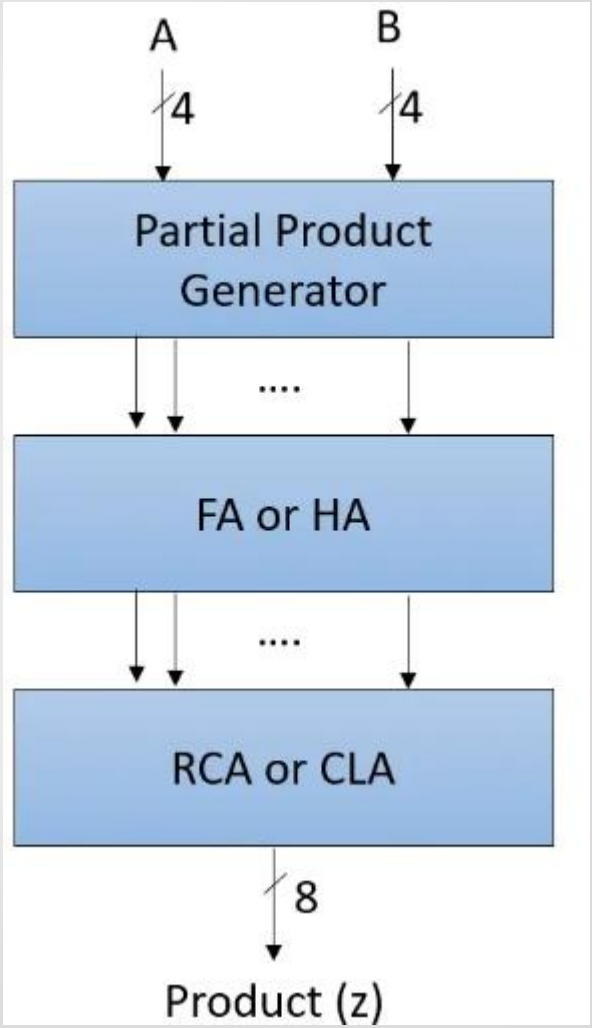
Full-Tree Multiplier divides the multiplication into:

1. Partial Products Calculation.
2. Partial products reduction tree, HAs & FAs.
3. Convert the redundant to binary.



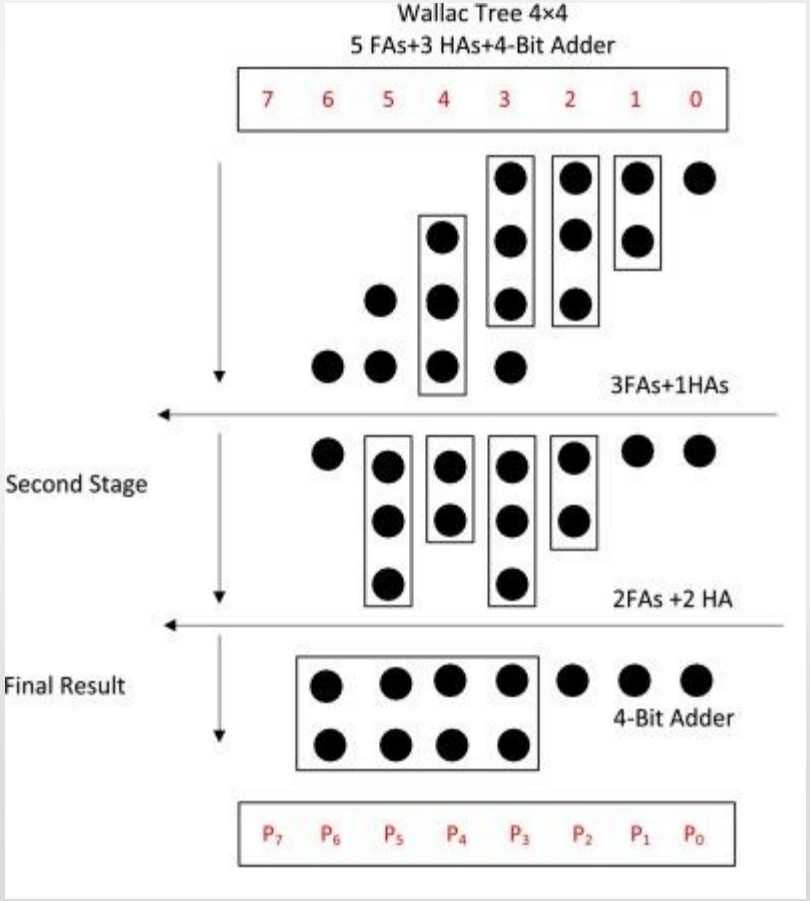
11.1 Full-Tree Multipliers

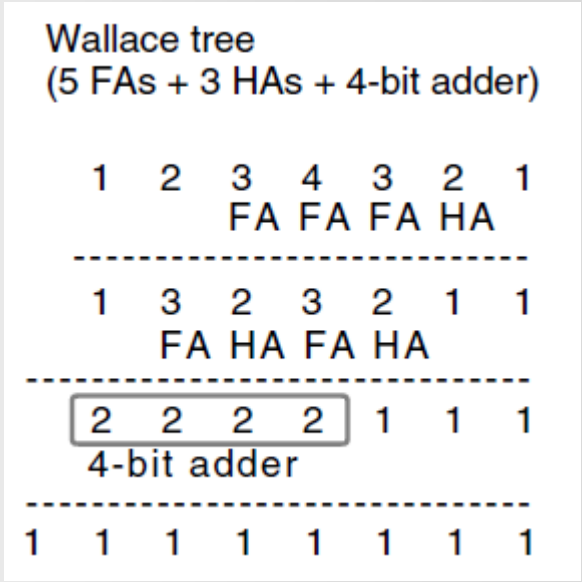
			a_3	a_2	a_1	a_0	
			b_3	b_2	b_1	b_0	
			p_{30}	p_{20}	p_{10}	p_{00}	
		p_{31}	p_{21}	p_{11}	p_{01}	x	
	p_{32}	p_{22}	p_{12}	p_{02}	x	x	
p_{33}	p_{23}	p_{13}	p_{03}	x	x	x	
z_7	z_6	z_5	z_4	z_3	z_2	z_1	z_0



11.1 Full-Tree Multipliers

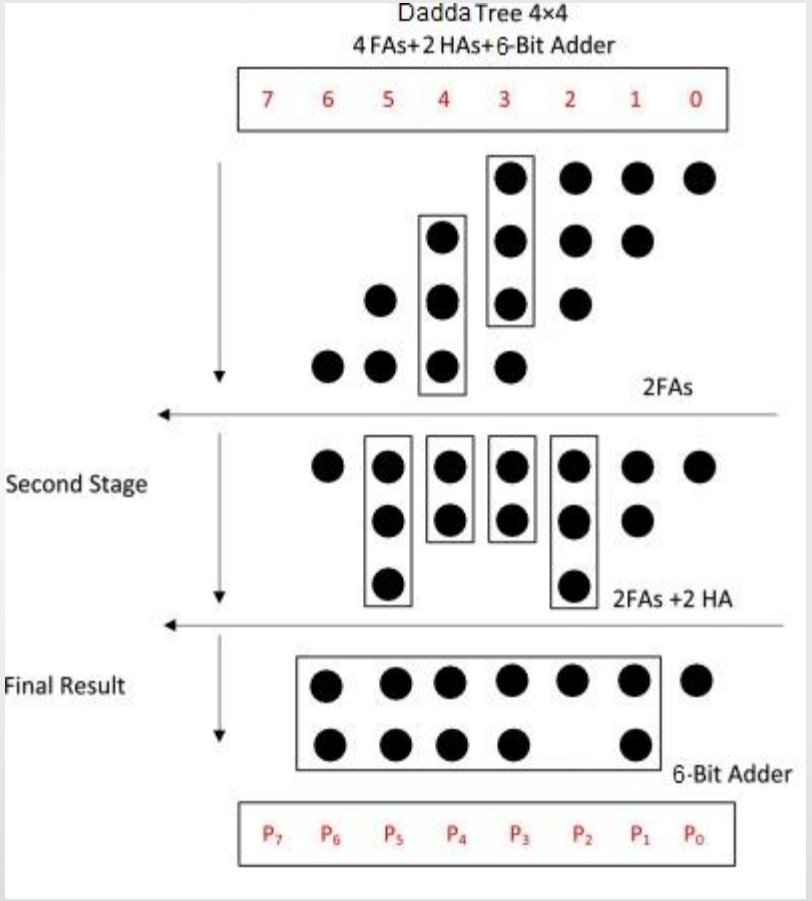
- **Wallace Tree Multiplier:**

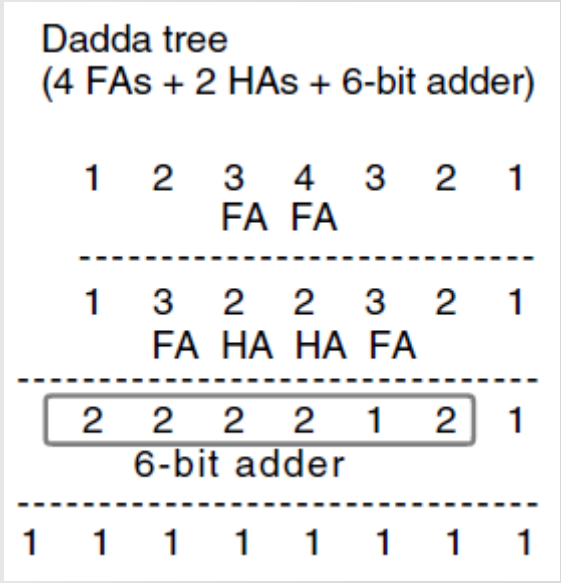




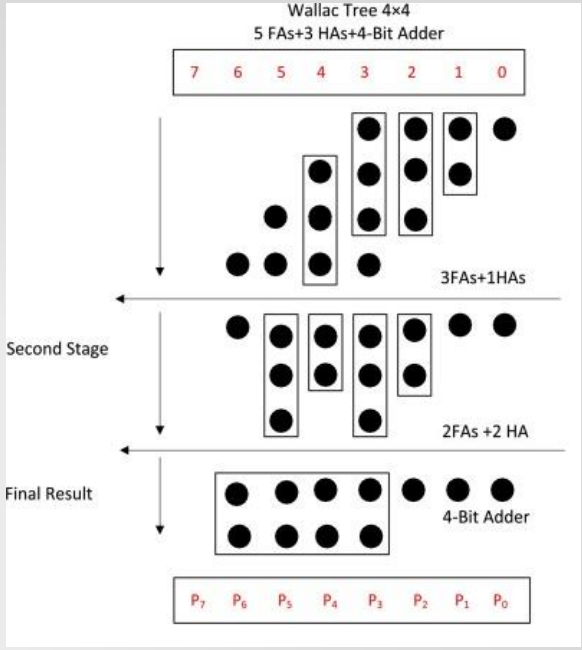
11.1 Full-Tree Multipliers

- **<u>Dadda Multiplier:</u>**
- To save more resources.





8.3 WALLACE AND DADDA TREES



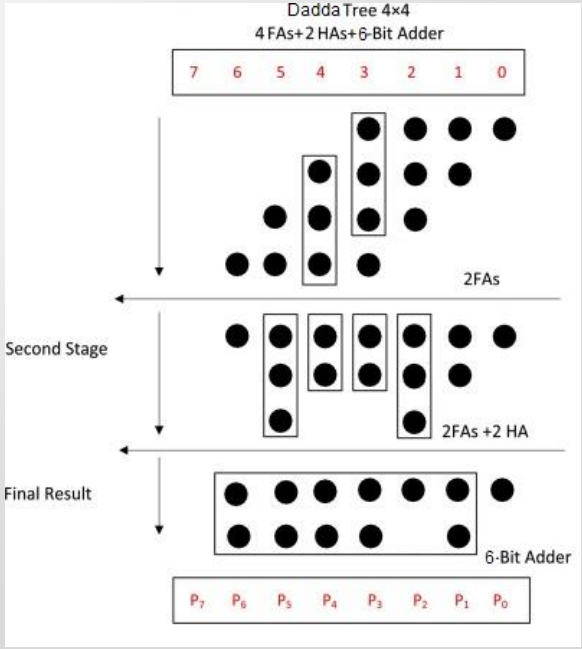
To calculate the number of steps:

1. $n_{j+1} = 2*\text{floor}(\frac{n_j}{3}) + n_j \bmod 3$, Starting with $n_0 = N$.

- If N=4, n_0=4, n_1=3, n_2=2.
- If N=8, n_0=8, n_1=6, n_2=4 , n_3=3 , n_4=2.

2. $n_{h+1} = \text{floor}(3/2*n_h)$, Starting with $n_0 = 2$

- $n_1 = \text{floor}(3/2*2) = 3$
- n_{h+1} = 2, 3, 4, 6, 9, 13, 19, 28, …

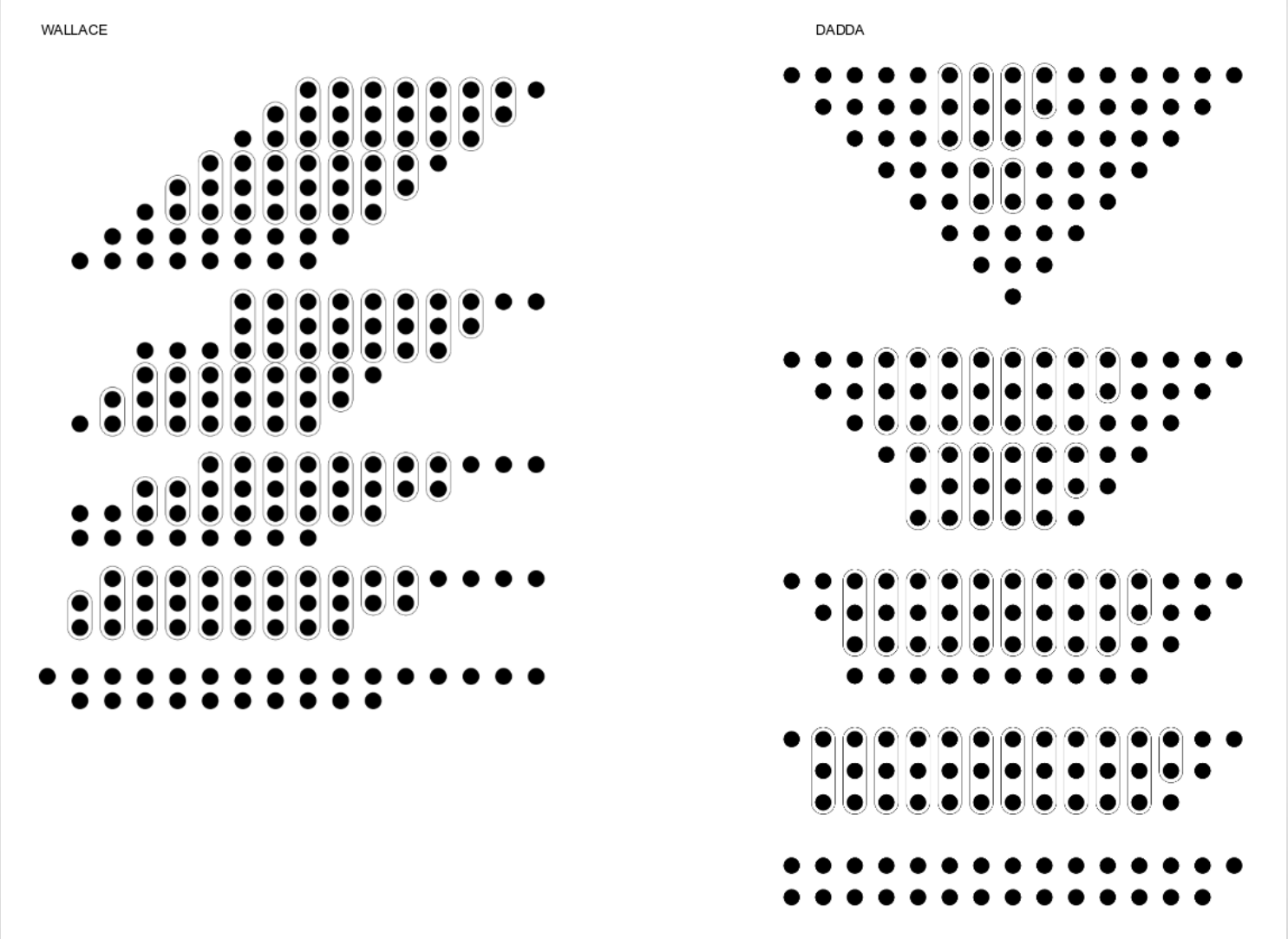


8.3 WALLACE AND DADDA TREES

- Then, we can use the following table directly
- We should choose n_1<min(N,M).
  - If we will multiply 5-bit*8-bits: n_0=5, n_1=4, n_2=3 , n_3=2.
  - If we will multiply 8-bit*8-bits: n_0=8, n_1=6, n_2=4, n_3=3 , n_4=2.

h	$n(h)$	h	$n(h)$	h	$n(h)$
0	2	7	28	14	474
1	3	8	42	15	711
2	4	9	63	16	1066
3	6	10	94	17	1599
4	9	11	141	18	2398
5	13	12	211	19	3597
6	19	13	316	20	5395

11.1 Full-Tree Multipliers



Extra References for Multiplier Algorithm

Wallace tree Multiplier :-

https://www.youtube.com/watch?v=75JidRFYAQ0

Dadda Multiplier :-

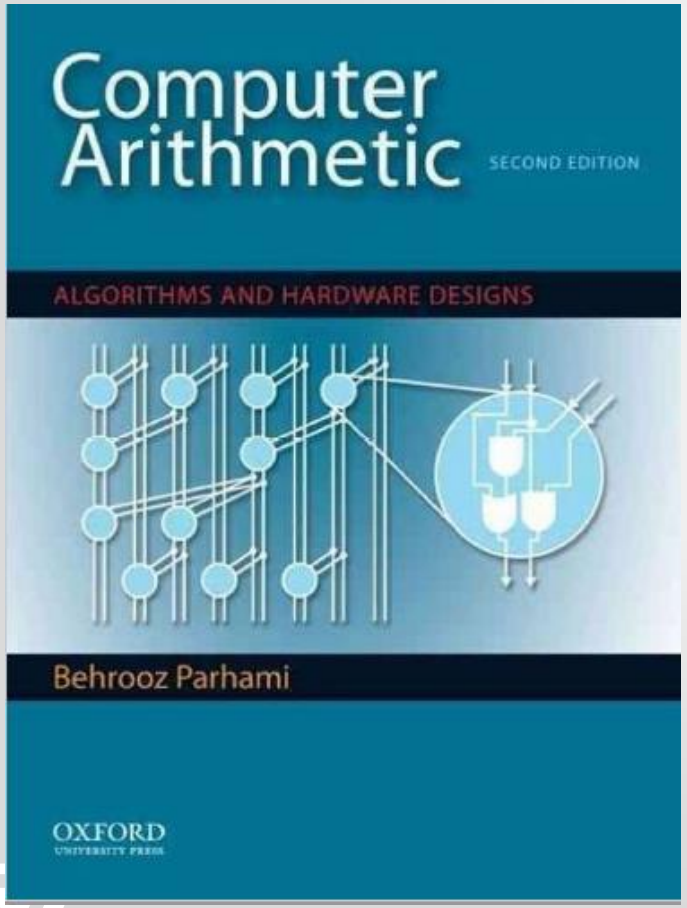
https://www.youtube.com/watch?v=Qd5DRrozvuY

Booth Multiplier :-

https://www.youtube.com/watch?v=C_PbgHDfeu8

Dr. Karim Abbas:

2. Divider



life.augmented

Agenda

13 Basic Division Schemes

- # 13.1 Shift/Subtract Division Algorithms
- # 13.3 Restoring Hardware Dividers
- # 13.4 Nonrestoring and Signed Division

Division Algorithms in Computer Arithmetic

1. **Restoring Division:**
   It involves restoring the dividend after each subtraction step to perform division.
2. **Non-Restoring Division:**
   It does not restore the dividend after each subtraction, making it more efficient than restoring division.
3. **SRT Division (Sweeney, Robertson, and Tocher Division):**
   1. SRT division is a high-speed division algorithm that uses a precomputed table to perform division.
   2. It is commonly used in high-performance processors.
4. **Newton-Raphson Division:**
   1. Newton-Raphson division is an iterative algorithm based on Newton's method for finding successively better approximations to the roots of a real-valued function.
   2. It can be used for division by reciprocal multiplication.

Division Algorithms in Computer Arithmetic

5.Goldschmidt Division:

1. Goldschmidt division is an iterative algorithm that uses reciprocal approximation to perform division.
2. It is known for its high accuracy and convergence speed.

6.Division by Multiplication:

1. Division by multiplication is a technique that involves multiplying the dividend by the reciprocal of the divisor to perform division.
2. It is commonly used in hardware implementations for efficient division.

7.Digit Recurrence Division:

1. Digit recurrence division is an algorithm that uses a set of precomputed values to perform division.
2. It is often used in hardware implementations for efficient division.

13.1 Shift/Subtract Division Algorithms

Example:

- Divide 117 ÷ 10 = 17 rem 10.

```
                          0  1  0  1  1      Quotient
Divisor   1  0  1  0 | 0  1  1  1  0  1  0  1   Dividend
                  -     1  0  1  0  0  0  0
                    ------------------------
                      0  1  0  0  1  0  1
                   -     0  0  0  0  0  0
                    ------------------------
                      0  1  0  0  1  0  1
                   -     1  0  1  0  0
                    ------------------------
                         1  0  0  0  1      Remainder
                      -     1  0  1  0
                       ---------------
                         0  1  1  1
```

(a) Integer division

z			0	1	1	1	0	1	0	1
2^4d			1	0	1	0				
$s^{(0)}$			0	1	1	1	0	1	0	1
$2s^{(0)}$	0	1	1	1	0	1	0	1		
$-q_3 2^4d$			1	0	1	0		$\{q_3 = 1\}$		
$s^{(1)}$			0	1	0	0	1	0	1	
$2s^{(1)}$	0	1	0	0	1	0	1			
$-q_2 2^4d$			0	0	0	0		$\{q_2 = 0\}$		
$s^{(2)}$			1	0	0	1	0	1		
$2s^{(2)}$	1	0	0	1	0	1				
$-q_1 2^4d$			1	0	1	0		$\{q_1 = 1\}$		
$s^{(3)}$			1	0	0	0	1			
$2s^{(3)}$	1	0	0	0	1					
$-q_0 2^4d$			1	0	1	0		$\{q_0 = 1\}$		
$s^{(4)}$			0	1	1	1				
s							0	1	1	1
q							1	0	1	1

13.3 Restoring Hardware Dividers

- How does hardware know if division fits?
  - Condition: if remainder ≥ divisor
  - Use subtraction: (remainder – divisor) ≥ 0
- OK, so if it fits, what do we do?
  - $\text{Remainder}_{n+1} = \text{Remainder}_{n} - \text{divisor}$
- What if it doesn't fit?
  - Have to restore original remainder
- Called restoring division [13.3 in the textbook]

13.3 Restoring Hardware Dividers

- At the start of each cycle j, the partial remainder $s^{(j-1)}$ is shifted to the left, with its MSB moving into a special flip-flop.
- Then, compute the trial difference $2s^{(j-1)} - q_{k-j}(2^k d)$. The next quotient digit should be 1 if the MSB of $2s^{(j-1)}$ is 1 or if the trial difference is positive ($c_{out} = 1$).
- If $q_{k-j} = 1$, the trial difference will be loaded into the upper half of the partial remainder register to form the new partial remainder for the next cycle.
- If $q_{k-j} = 0$, the partial remainder does not change.

z		0	1	1	1	0	1	0	1
$2^4 d$	0	1	0	1	0				
$-2^4 d$	1	0	1	1	0				
$s^{(0)}$	0	0	1	1	1	0	1	0	1
$2s^{(0)}$	0	1	1	1	0	1	0	1	
$+(-2^4 d)$	1	0	1	1	0				
$s^{(1)}$	0	0	1	0	0	1	0	1	
$2s^{(1)}$	0	1	0	0	1	0	1		
$+(-2^4 d)$	1	0	1	1	0				
$s^{(2)}$	1	1	1	1	1	0	1		
$s^{(2)} = 2s^{(1)}$	0	1	0	0	1	0	1		
$2s^{(2)}$	1	0	0	1	0	1			
$+(-2^4 d)$	1	0	1	1	0				
$s^{(3)}$	0	1	0	0	0	1			
$2s^{(3)}$	1	0	0	0	1				
$+(-2^4 d)$	1	0	1	1	0				
$s^{(4)}$	0	0	1	1	1				
s						0	1	1	1
q						1	0	1	1

No overflow, since: $(0111)_{two} < (1010)_{two}$

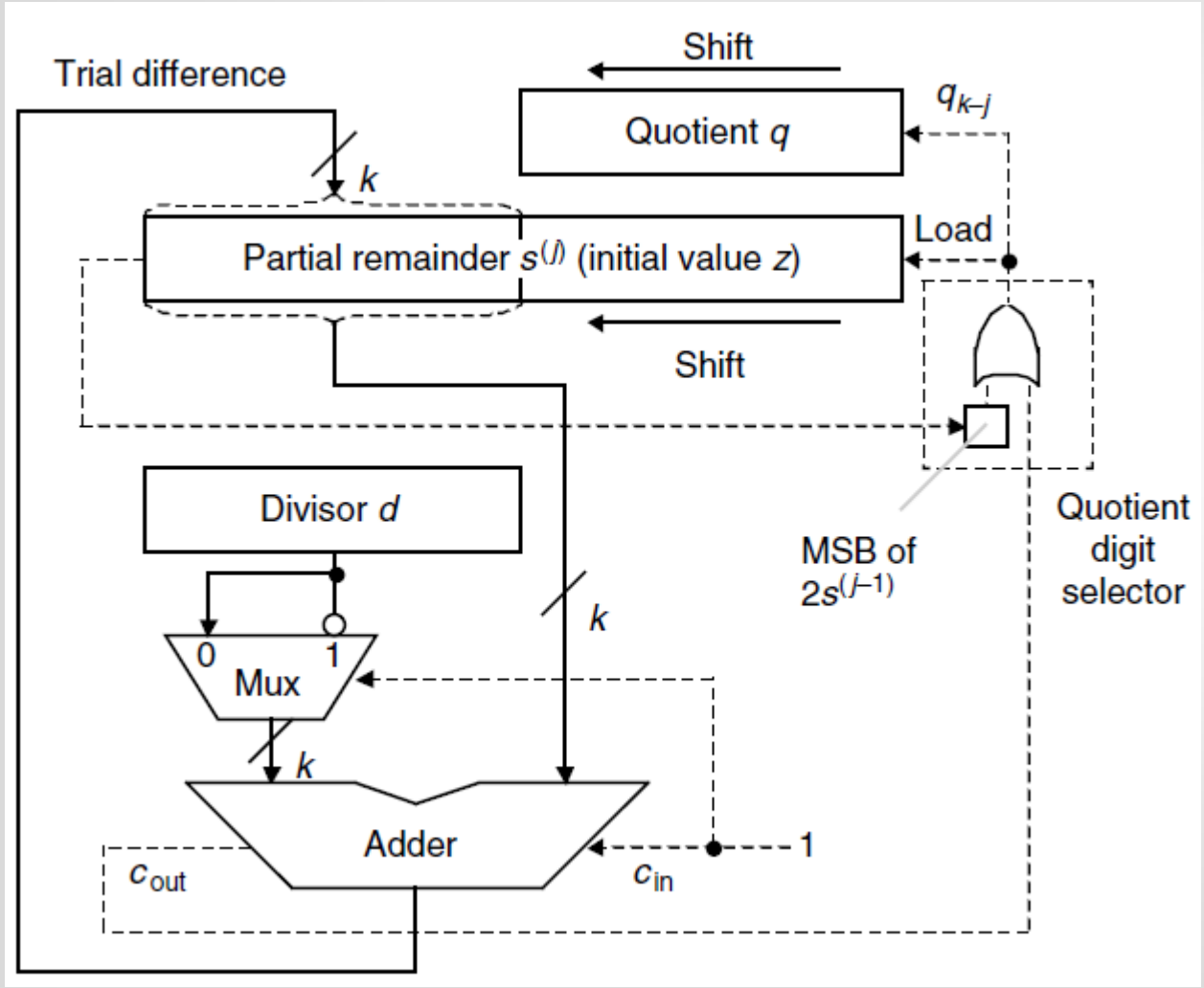
Positive, so set $q_3 = 1$

Negative, so set $q_2 = 0$ and restore

Positive, so set $q_1 = 1$

Positive, so set $q_0 = 1$

13.3 Restoring Hardware Dividers



z		0	1	1	1	0	1	0	1
2^4d	0	1	0	1	0				
-2^4d	1	0	1	1	0				
$s^{(0)}$	0	0	1	1	1	0	1	0	1
$2s^{(0)}$	0	1	1	1	0	1	0	1	
$+(-2^4d)$	1	0	1	1	0				
$s^{(1)}$	0	0	1	0	0	1	0	1	
$2s^{(1)}$	0	1	0	0	1	0	1		
$+(-2^4d)$	1	0	1	1	0				
$s^{(2)}$	1	1	1	1	1	0	1		
$s^{(2)} = 2s^{(1)}$	0	1	0	0	1	0	1		
$2s^{(2)}$	1	0	0	1	0	1			
$+(-2^4d)$	1	0	1	1	0				
$s^{(3)}$	0	1	0	0	0	1			
$2s^{(3)}$	1	0	0	0	1				
$+(-2^4d)$	1	0	1	1	0				
$s^{(4)}$	0	0	1	1	1				
s						0	1	1	1
q						1	0	1	1

No overflow, since: $(0111)_{two} < (1010)_{two}$

Positive, so set $q_3 = 1$

Negative, so set $q_2 = 0$ and restore

Positive, so set $q_1 = 1$

Positive, so set $q_0 = 1$

13.3 Restoring Hardware Dividers

- Examples of integer division with all possible combinations of signs for z and d:
  - $z = 5\ d = 3 \Rightarrow q = 1\ s = 2$
  - $z = 5\ d = -3 \Rightarrow q = -1\ s = 2$
  - $z = -5\ d = 3 \Rightarrow q = -1\ s = -2$
  - $z = -5\ d = -3 \Rightarrow q = 1\ s = -2$
- The magnitudes of q and s are unaffected by the input signs and that the signs of q and s are easily derivable from the signs of z and d.
- An added indirect algorithm converts the operands into unsigned values and, at the end, accounts for the signs by adjusting the sign bits or via complementation.

13.4 Nonrestoring and Signed Division

- Division still takes:
  - 2 ALU cycles per bit position
    - 1 to check for divisibility (subtract)
    - One to restore (if needed)
- Can reduce to 1 cycle per bit
  - Called non-restoring division
  - Avoids restore of remainder when test fails

13.4 Nonrestoring and Signed Division

- Suppose that the shifted partial remainder at the start of the cycle was u.
- If we had restored the partial remainder u -2^kd to its correct value u, we would proceed with the next shift and trial subtraction, getting the result 2u-2^kd.
- If we used the incorrect partial remainder, it would yield 2(u-2^kd)-2^kd = 2u $-$ (3*2^kd).
- So, an addition can be performed, resulting in 2(u $-$ 2^kd) + 2^kd = 2u $-$ 2^kd, which is the same value obtained after restoration and trial subtraction.
- Thus, in non-restoring division, when the partial remainder becomes negative, we keep the incorrect partial remainder, but note the correct quotient digit and also remember to add, rather than subtract, in the next cycle.

z		0	1	1	1	0	1	0	1
2^4d	0	1	0	1	0				
-2^4d	1	0	1	1	0				
$s^{(0)}$	0	0	1	1	1	0	1	0	1
$2s^{(0)}$	0	1	1	1	0	1	0	1	
$+(-2^4d)$	1	0	1	1	0				
$s^{(1)}$	0	0	1	0	0	1	0	1	
$2s^{(1)}$	0	1	0	0	1	0	1		
$+(-2^4d)$	1	0	1	1	0				
$s^{(2)}$	1	1	1	1	1	0	1		
$2s^{(2)}$	1	1	1	1	0	1			
$+2^4d$	0	1	0	1	0				
$s^{(3)}$	0	1	0	0	0	1			
$2s^{(3)}$	1	0	0	0	1				
$+(-2^4d)$	1	0	1	1	0				
$s^{(4)}$	0	0	1	1	1				
s						0	1	1	1
q						1	0	1	1

No overflow, since: $(0111)_{two} < (1010)_{two}$

Positive, so subtract

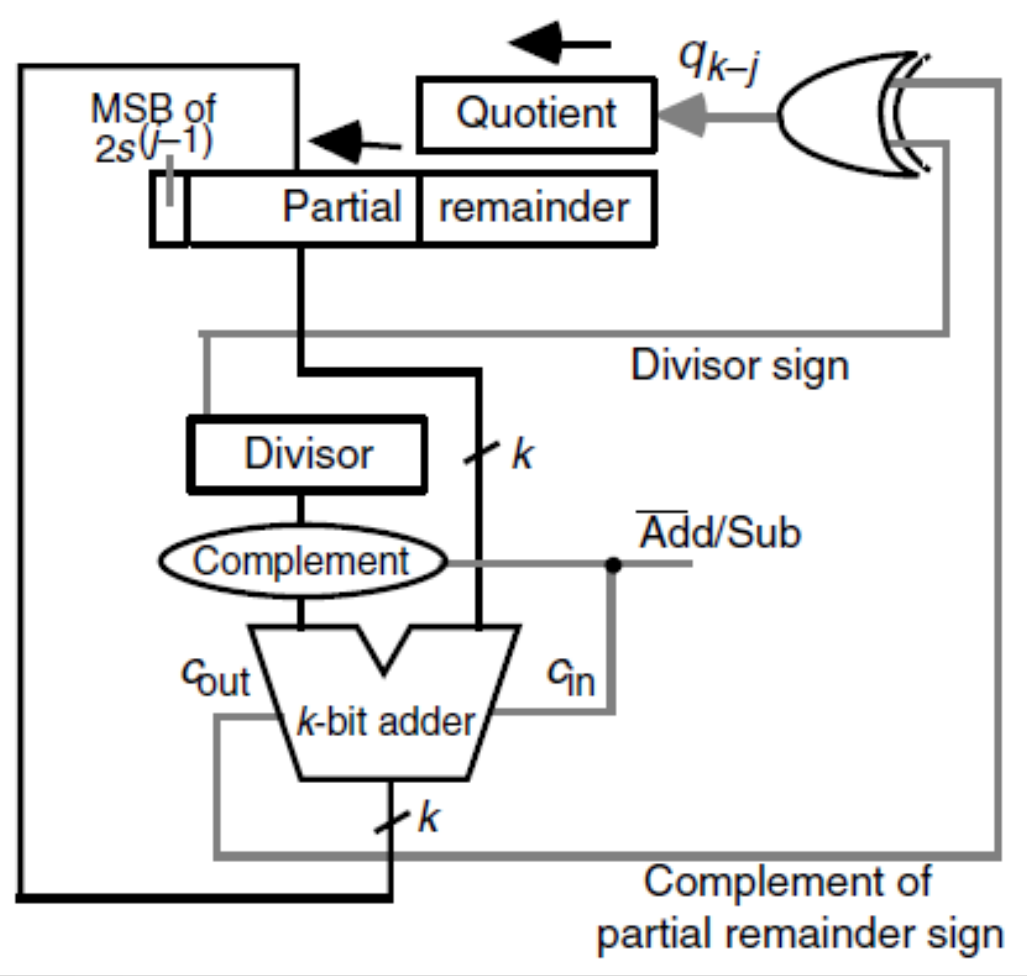
Positive, so set $q_3 = 1$ and subtract

Negative, so set $q_2 = 0$ and add

Positive, so set $q_1 = 1$ and subtract

Positive, so set $q_0 = 1$

13.4 Nonrestoring and Signed Division



z		0	1	1	1	0	1	0	1
2^4d	0	1	0	1	0				
-2^4d	1	0	1	1	0				
$s^{(0)}$	0	0	1	1	1	0	1	0	1
$2s^{(0)}$	0	1	1	1	0	1	0	1	
$+(-2^4d)$	1	0	1	1	0				
$s^{(1)}$	0	0	1	0	0	1	0	1	
$2s^{(1)}$	0	1	0	0	1	0	1		
$+(-2^4d)$	1	0	1	1	0				
$s^{(2)}$	1	1	1	1	1	0	1		
$2s^{(2)}$	1	1	1	1	0	1			
$+2^4d$	0	1	0	1	0				
$s^{(3)}$	0	1	0	0	0	1			
$2s^{(3)}$	1	0	0	0	1				
$+(-2^4d)$	1	0	1	1	0				
$s^{(4)}$	0	0	1	1	1				
s						0	1	1	1
q						1	0	1	1

No overflow, since: $(0111)_{two} < (1010)_{two}$

Positive, so subtract

Positive, so set $q_3 = 1$ and subtract

Negative, so set $q_2 = 0$ and add

Positive, so set $q_1 = 1$ and subtract

Positive, so set $q_0 = 1$

13.4 Nonrestoring and Signed Division

Iteration	Step	Divisor	Remainder
0	Initial values	0010	0000 0111
	Shift rem left 1	0010	0000 1110
1	2: Rem = Rem - Div	0010	1110 1110
	3b: Rem < 0 (add next), sll 0	0010	1101 1100
2	2: Rem = Rem + Div	0010	1111 1100
	3b: Rem < 0 (add next), sll 0	0010	1111 1000
3	2: Rem = Rem + Div	0010	0001 1000
	3a: Rem > 0 (sub next), sll 1	0010	0011 0001
4	Rem = Rem – Div	0010	0001 0001
	Rem > 0 (sub next), sll 1	0010	0010 0011
	Shift Rem right by 1	0010	0001 0011

Summary

- Integer dividers covered
  - Multicycle restoring
  - Non-restoring
- Other approaches
  - SRT division [Sweeney, Robertson, Tocher] uses lookup tables
    - Famous Intel fdiv bug caused by incomplete table
  - Newton-Raphson method
    - Estimate reciprocal, iterate to refine, multiply
  - Beyond the scope of this course

3. Implementation

Verilog Code

```
module alu ( input [31:0] a, b;
...
);
wire [31:0] a_plus_b;
//wire signed[31:0] a_plus_b_signed;

assign a_plus_b = a+b;  //Add 1011+1100 = +7
//assign a_plus_b_signed = $singed(a+b);  // +7

assign a_mul = (mul_signed&a[31])?-a:a;
assign b_mul = (mul_signed&b[31])?-b:b;
assign y_mul = a_mul*b_mul;
assign y = (mul_signed&a[31]&~b[31]|mul_signed&~a[31]&b[31])?-y_mul:y_mul;

case (alu_control)
        add_signed: begin  alu_result = a_plus_b;
                overflow = (a[31]&b[31]&~a_plus_b[31])|(~a[31]&~b[31]&a_plus_b[31]);     end
        add_unsigned: alu_result = a_plus_b;  overflow =0;
        mul_signed: alu_result =y;
        mul_unsigned: alu_result =y;  //alu_result =y_mul
        ...
endcase

endmodule
```

4. Synthesis

Thank You